

01_antonelli_vs_russell

May 13, 2026

1 Antonelli vs Russell — 2026 Qualifying, First Four Rounds

I wanted to see where Kimi Antonelli's 2026 season at Mercedes actually stands vs his teammate George Russell. This notebook compares both Mercedes drivers' fastest qualifying laps across the first four rounds of 2026, broken down by track segment. See the [project README](#) for full method and limitations.

```
[1]: %matplotlib inline
import sys
sys.path.insert(0, '..')
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from src.loaders import setup_cache, load_qualifying_session,
    ↳get_fastest_valid_lap, get_lap_telemetry
from src.segments import resample_to_distance_grid
from src.benchmarks import compare_teammates, compute_corner_signatures
from src.plotting import plot_category_deltas, plot_lap_delta_by_round,
    ↳plot_track_delta_map, plot_yoy_lap_deltas, plot_corner_bucket_summary,
    ↳plot_corner_signatures

setup_cache('../fastf1_cache')
RACES = ['Australia', 'China', 'Japan', 'Miami']
```

1.1 Method (briefly)

For each race: load qualifying, take each driver's fastest valid lap, and segment the track from FastF1's `circuit_info.corners` (corners within 250 m are grouped, straights are filled in between). Telemetry — including FastF1's per-sample `Time` channel — is resampled onto a uniform 5 m distance grid. Per-segment time is read directly from that resampled `Time` channel as `Time[last_step] - Time[first_step]` inside the segment. Per-segment delta is `time(Russell) - time(Antonelli)`, so **positive = Antonelli faster**.

An earlier version integrated `step / speed` per grid point. On 2 of 4 races the accumulated residual exceeded the 0.1 s sanity-check threshold (see the next section), so the analysis switched to reading FastF1's sample times directly. Residuals are now `< 0.1 s` on all four races.

Each segment is classified into one of four categories by its minimum speed: slow corner (<130 kph), medium corner (130–200 kph), fast corner (>200 kph), or straight.

```
[2]: results = {race: compare_teammates(2026, race) for race in RACES}

meta_df = pd.DataFrame([results[r]['meta'] for r in RACES])
summary = meta_df[['round', 'event_name', 'lap_time_a_s', 'lap_time_b_s',
    ↳ 'lap_delta_s', 'q_session_a', 'q_session_b', 'q_mismatch']]
summary.columns = ['Round', 'Race', 'ANT lap (s)', 'RUS lap (s)', '\u0394 lap_
    ↳ (s)', 'ANT Q', 'RUS Q', 'Q mismatch']
summary
```

```
core          INFO      Loading data for Australian Grand Prix - Qualifying
[v3.8.1]

req           INFO      Using cached data for session_info

req           INFO      Using cached data for driver_info

req           INFO      Using cached data for session_status_data

req           INFO      Using cached data for track_status_data

req           INFO      Using cached data for _extended_timing_data

req           INFO      Using cached data for timing_app_data

core          INFO      Processing timing data...

core          WARNING    No lap data for driver 18

core          WARNING    No lap data for driver 55

core          WARNING    Failed to perform lap accuracy check - all laps marked
as inaccurate (driver 18)

core          WARNING    Failed to perform lap accuracy check - all laps marked
as inaccurate (driver 55)

req           INFO      Using cached data for car_data

req           INFO      Using cached data for position_data

req           INFO      Using cached data for weather_data

req           INFO      Using cached data for race_control_messages

core          INFO      Finished loading data for 22 drivers: ['63', '12', '6',
'16', '81', '1', '44', '30', '41', '5', '27', '87', '31', '10', '23', '43',
'14', '11', '77', '18', '3', '55']

core          INFO      Loading data for Chinese Grand Prix - Qualifying
[v3.8.1]

req           INFO      Using cached data for session_info

req           INFO      Using cached data for driver_info
```

```

req          INFO      Using cached data for session_status_data
req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 22 drivers: ['12', '63', '44',
'16', '81', '1', '10', '3', '6', '87', '27', '43', '31', '30', '41', '5', '55',
'23', '14', '77', '18', '11']
core         INFO      Loading data for Japanese Grand Prix - Qualifying
[v3.8.1]
req          INFO      Using cached data for session_info
req          INFO      Using cached data for driver_info
req          INFO      Using cached data for session_status_data
req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 22 drivers: ['12', '63', '81',
'16', '1', '44', '10', '6', '5', '41', '3', '31', '27', '30', '43', '55', '23',
'87', '11', '77', '14', '18']
core         INFO      Loading data for Miami Grand Prix - Qualifying [v3.8.1]
req          INFO      Using cached data for session_info
req          INFO      Using cached data for driver_info
req          INFO      Using cached data for session_status_data
req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data

```

```

req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 22 drivers: ['12', '3', '16',
'1', '63', '44', '81', '43', '10', '27', '30', '87', '55', '31', '23', '41',
'14', '18', '77', '11', '5', '6']

```

```

[2]:
Round      Race  ANT lap (s)  RUS lap (s)  Δ lap (s)  ANT Q  \
0         1  Australian Grand Prix    78.811    78.518    -0.293    3
1         2    Chinese Grand Prix    92.064    92.286     0.222    3
2         3   Japanese Grand Prix    88.778    89.076     0.298    3
3         4    Miami Grand Prix    87.798    88.197     0.399    3

RUS Q  Q mismatch
0      3      False
1      3      False
2      3      False
3      3      False

```

```

[3]: mismatch_races = [r['meta']['event_name'] for r in results.values() if
    ↳r['meta']['q_mismatch']]
unknown_races = [r['meta']['event_name'] for r in results.values() if
    ↳r['meta']['q_session_a'] is None or r['meta']['q_session_b'] is None]
if mismatch_races:
    print('\u26a0 Q-session mismatch \u2014 drivers\' fastest valid laps came
    ↳from different Q-segments at:')
    for race in mismatch_races:
        meta = next(r['meta'] for r in results.values() if
    ↳r['meta']['event_name'] == race)
        print(f"    {race}: ANT Q{meta['q_session_a']} vs RUS
    ↳Q{meta['q_session_b']} \u2014 track evolution likely confounds this delta")
if unknown_races:
    print('\u26a0 Q-session could not be determined for:', ', ', '.
    ↳join(unknown_races))
if not mismatch_races and not unknown_races:
    print('No Q-session mismatches across the 4 races.')

```

No Q-session mismatches across the 4 races.

1.2 Sensor quality

FastF1 telemetry doesn't carry an explicit quality flag, but a frozen **Speed** sensor produces a clear signature: only a handful of unique speed values across many samples within one segment. Any segment that fails this check is excluded from the headline category analysis below — the per-segment time delta is also unreliable in those segments because cumulative **Distance** is integrated from the bad speed.

```
[4]: flagged = []
for r in results.values():
    bad = r['segments'][~r['segments']['sensor_ok']]
    for row in bad.itertuples():
        flagged.append({
            'race': r['meta']['event_name'],
            'segment': row.Segment, 'kind': row.kind, 'category': row.category,
            'min_speed_kph': round(row.min_speed, 1),
            'delta_s_pre_filter': round(row.delta_s, 3),
        })
if flagged:
    print(f"Flagged {len(flagged)} segment(s) where a driver's speed sensor_
↳looks frozen:")
    display(pd.DataFrame(flagged))
else:
    print('No sensor-freeze flags across the 4 races.')
```

Flagged 5 segment(s) where a driver's speed sensor looks frozen:

	race	segment	kind	category	min_speed_kph	\
0	Japanese Grand Prix	S7	straight	straight	177.2	
1	Japanese Grand Prix	C7	corner	medium_corner	189.0	
2	Japanese Grand Prix	S8	straight	straight	189.0	
3	Japanese Grand Prix	C8	corner	medium_corner	189.0	
4	Japanese Grand Prix	S9	straight	straight	97.0	

	delta_s_pre_filter
0	-3.555
1	-0.766
2	-1.663
3	-2.416
4	8.712

1.3 Sanity check

The segmentation logic is only useful if the segment deltas sum back to the actual lap-time delta. The table below verifies this on real telemetry: any $|\Delta|$ above 0.1 s would indicate a bug.

```
[5]: sanity = []
for race, r in results.items():
    sum_deltas = float(r['segments']['delta_s'].sum())
```

```

lap_delta = float(r['meta']['lap_delta_s'])
sanity.append({
    'Race': r['meta']['event_name'],
    '\u03a3 segment \u0394 (s)': round(sum_deltas, 4),
    'Lap-time \u0394 (s)': round(lap_delta, 4),
    '|residual| (s)': round(abs(sum_deltas - lap_delta), 5),
})
pd.DataFrame(sanity)

```

```

[5]:
      Race  Σ segment Δ (s)  Lap-time Δ (s)  |residual| (s)
0  Australian Grand Prix      -0.3838      -0.293      0.09084
1    Chinese Grand Prix       0.2352       0.222      0.01318
2   Japanese Grand Prix       0.2758       0.298      0.02216
3    Miami Grand Prix        0.3758       0.399      0.02316

```

```

[6]: cat_check = pd.concat([
    r['segments'].assign(event_name=r['meta']['event_name'])
    for r in results.values()
])
cat_check.groupby(['event_name', 'category']).size().unstack(fill_value=0)

```

```

[6]: category          fast_corner  medium_corner  slow_corner  straight
event_name
Australian Grand Prix           3             1             3           8
Chinese Grand Prix              2             1             5           9
Japanese Grand Prix             2             5             1           9
Miami Grand Prix                2             0             4           7

```

1.4 Headline finding — where does the time go?

Each bar is the mean per-category delta across all four races. Black dots show each individual race's per-category mean — they tell you whether the average reflects a consistent pattern or a noisy one.

```

[7]: all_segs = pd.concat([
    r['segments'].assign(event_name=r['meta']['event_name'])
    for r in results.values()
], ignore_index=True)

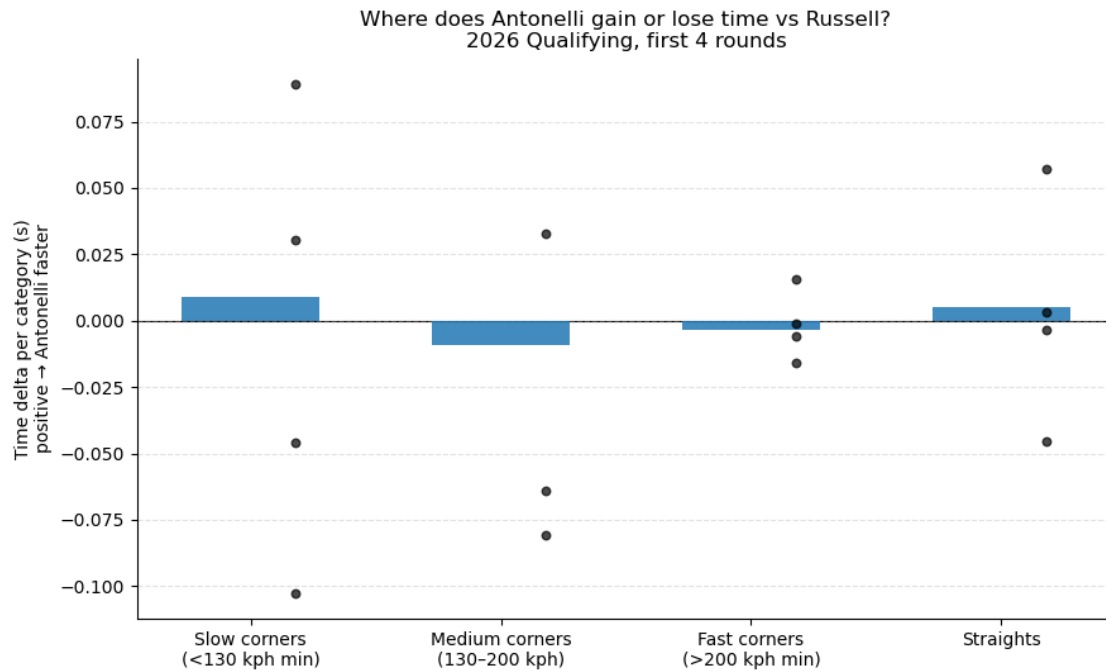
# Exclude segments where either driver's speed sensor looks frozen - in those
# segments min_speed (and therefore the category label) is unreliable, and the
# per-segment delta itself can be wildly distorted by integrated-Distance error.
# See the "Sensor quality" cell below the summary table for which segments
# were dropped and why.
clean = all_segs[all_segs['sensor_ok']].copy()
n_excluded = len(all_segs) - len(clean)
print(f"Headline chart uses {len(clean)} segments after excluding {n_excluded}
↳with sensor-freeze flags.")

```

```
fig = plot_category_deltas(clean, save_path='../figures/headline_segment_delta.
    png')
plt.show()

# Print the post-filter per-category means for the prose interpretation below
print("\nPer-category means after filtering (s/lap, positive = ANT faster):")
print(clean.groupby('category')['delta_s'].mean().round(4).to_string())
```

Headline chart uses 57 segments after excluding 5 with sensor-freeze flags.



Per-category means after filtering (s/lap, positive = ANT faster):

```
category
fast_corner      -0.0034
medium_corner    -0.0093
slow_corner       0.0088
straight         0.0051
```

The segment-level picture, after excluding five Japan segments where Antonelli's speed sensor was frozen for ~1300 m of the lap, is quieter than I expected:

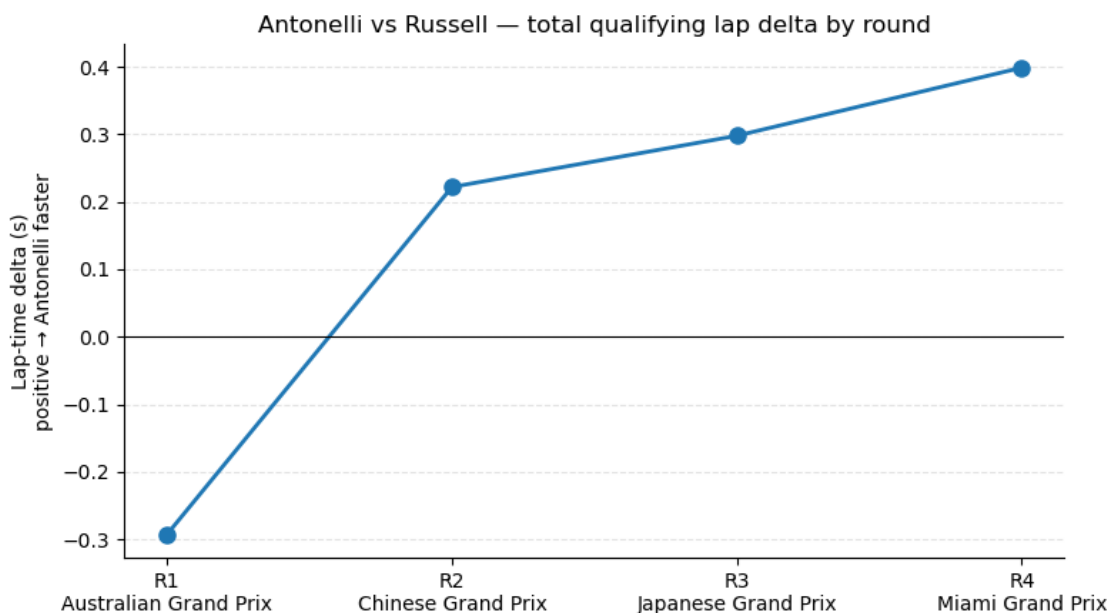
- **Straights:** essentially tied (+0.005 s/lap).
- **Slow corners (<130 kph):** essentially tied (+0.009 s/lap).
- **Fast corners (>200 kph):** essentially tied (-0.003 s/lap).
- **Medium corners (130-200 kph):** essentially tied (-0.009 s/lap).

Earlier versions of this analysis (before the sensor-quality filter was tightened) reported large per-category deltas — “+0.17 s/lap on straights” and “−0.46 s/lap in medium corners.” Both numbers turned out to be driven almost entirely by five segments at Japan where the speed sensor was stuck near 189 kph. Once those segments are removed, no single category emerges as the source of Antonelli’s lap-time advantage; the +0.16 s/lap mean over four races is spread across the whole lap rather than concentrated in any one phase.

The cleaner read: at four races and with one of them partially compromised, the trajectory finding (next section) is what the data actually supports.

1.5 Trajectory — is the gap closing?

```
[8]: fig = plot_lap_delta_by_round(meta_df, save_path='../figures/lap_delta_by_round.
      ↪png')
      plt.show()
```



Across four rounds, the lap-time gap has gone from **−0.29 s** (Russell faster by 0.29 s) at Australia to **+0.40 s** (Antonelli faster by 0.40 s) at Miami — a monotone trend in Antonelli’s favour covering **0.69 s across four races**. Four data points is too few to project a trajectory with confidence, but the direction is striking: Antonelli was behind at Round 1 and ahead on every subsequent round, with the margin growing each time. Australia looks like the outlier round, not the norm.

1.6 Where on the track is each driver gaining time?

The category breakdown averages across each segment, but it can hide where exactly within a segment the time is found. Below, each circuit is shown as a track map coloured by the **local slope of the cumulative time delta** — the rate at which the gap is changing as the lap unfolds.

- **Blue** = Antonelli is gaining on Russell at that part of the track.

- **Red** = Russell is gaining on Antonelli.
- Corner numbers (from FastF1's `circuit_info`) are overlaid for orientation.

```
[9]: fig, axes = plt.subplots(2, 2, figsize=(13, 11))

# Precompute slopes for all 4 races so the color scale is shared across subplots
race_data = []
for race in RACES:
    session = load_qualifying_session(2026, race)
    lap_a = get_fastest_valid_lap(session, 'ANT')
    lap_b = get_fastest_valid_lap(session, 'RUS')
    res_a = resample_to_distance_grid(get_lap_telemetry(lap_a))
    res_b = resample_to_distance_grid(get_lap_telemetry(lap_b))
    n = min(len(res_a), len(res_b))
    # Cumulative delta(d) = Time_b(d) - Time_a(d); positive => ANT ahead at
    ↪ distance d
    delta = res_b['Time'].values[:n] - res_a['Time'].values[:n]
    # Local slope d(delta)/dd, smoothed over ~50 m to suppress sampling noise
    slope = np.gradient(delta)
    slope_smooth = pd.Series(slope).rolling(10, center=True, min_periods=1).
    ↪ mean().values
    race_data.append({
        'race': race,
        'x': res_a['X'].values[:n],
        'y': res_a['Y'].values[:n],
        'slope': slope_smooth,
        'corners': session.get_circuit_info().corners,
    })

# Shared symmetric color limit (95th percentile across all races)
all_slopes = np.concatenate([d['slope'] for d in race_data])
vmax = float(np.percentile(np.abs(all_slopes), 95))

last_mappable = None
for ax, d in zip(axes.ravel(), race_data):
    _, mappable = plot_track_delta_map(
        d['x'], d['y'], d['slope'], d['corners'],
        ax=ax, title=d['race'], vmax=vmax,
    )
    last_mappable = mappable

fig.suptitle(
    'Where Antonelli gains (blue) and where Russell gains (red)\nLocal rate of
    ↪ time gain/loss across the lap',
    fontsize=13,
)
```

```

cbar = fig.colorbar(last_mappable, ax=axes.ravel().tolist(), shrink=0.7, pad=0.
    ↪02,
                        orientation='horizontal', location='bottom')
cbar.set_label('d(time-delta)/d(distance) - negative: Russell gaining | ↪
    ↪positive: Antonelli gaining')
fig.savefig('../figures/track_delta_map.png', dpi=150, bbox_inches='tight')
plt.show()

```

```

core          INFO      Loading data for Australian Grand Prix - Qualifying
[v3.8.1]

req           INFO      Using cached data for session_info

req           INFO      Using cached data for driver_info

req           INFO      Using cached data for session_status_data

req           INFO      Using cached data for track_status_data

req           INFO      Using cached data for _extended_timing_data

req           INFO      Using cached data for timing_app_data

core          INFO      Processing timing data...

core          WARNING    No lap data for driver 18

core          WARNING    No lap data for driver 55

core          WARNING    Failed to perform lap accuracy check - all laps marked
as inaccurate (driver 18)

core          WARNING    Failed to perform lap accuracy check - all laps marked
as inaccurate (driver 55)

req           INFO      Using cached data for car_data

req           INFO      Using cached data for position_data

req           INFO      Using cached data for weather_data

req           INFO      Using cached data for race_control_messages

core          INFO      Finished loading data for 22 drivers: ['63', '12', '6',
'16', '81', '1', '44', '30', '41', '5', '27', '87', '31', '10', '23', '43',
'14', '11', '77', '18', '3', '55']

core          INFO      Loading data for Chinese Grand Prix - Qualifying
[v3.8.1]

req           INFO      Using cached data for session_info

req           INFO      Using cached data for driver_info

req           INFO      Using cached data for session_status_data

req           INFO      Using cached data for track_status_data

```

```

req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 22 drivers: ['12', '63', '44',
'16', '81', '1', '10', '3', '6', '87', '27', '43', '31', '30', '41', '5', '55',
'23', '14', '77', '18', '11']
core         INFO      Loading data for Japanese Grand Prix - Qualifying
[v3.8.1]
req          INFO      Using cached data for session_info
req          INFO      Using cached data for driver_info
req          INFO      Using cached data for session_status_data
req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 22 drivers: ['12', '63', '81',
'16', '1', '44', '10', '6', '5', '41', '3', '31', '27', '30', '43', '55', '23',
'87', '11', '77', '14', '18']
core         INFO      Loading data for Miami Grand Prix - Qualifying [v3.8.1]
req          INFO      Using cached data for session_info
req          INFO      Using cached data for driver_info
req          INFO      Using cached data for session_status_data
req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...

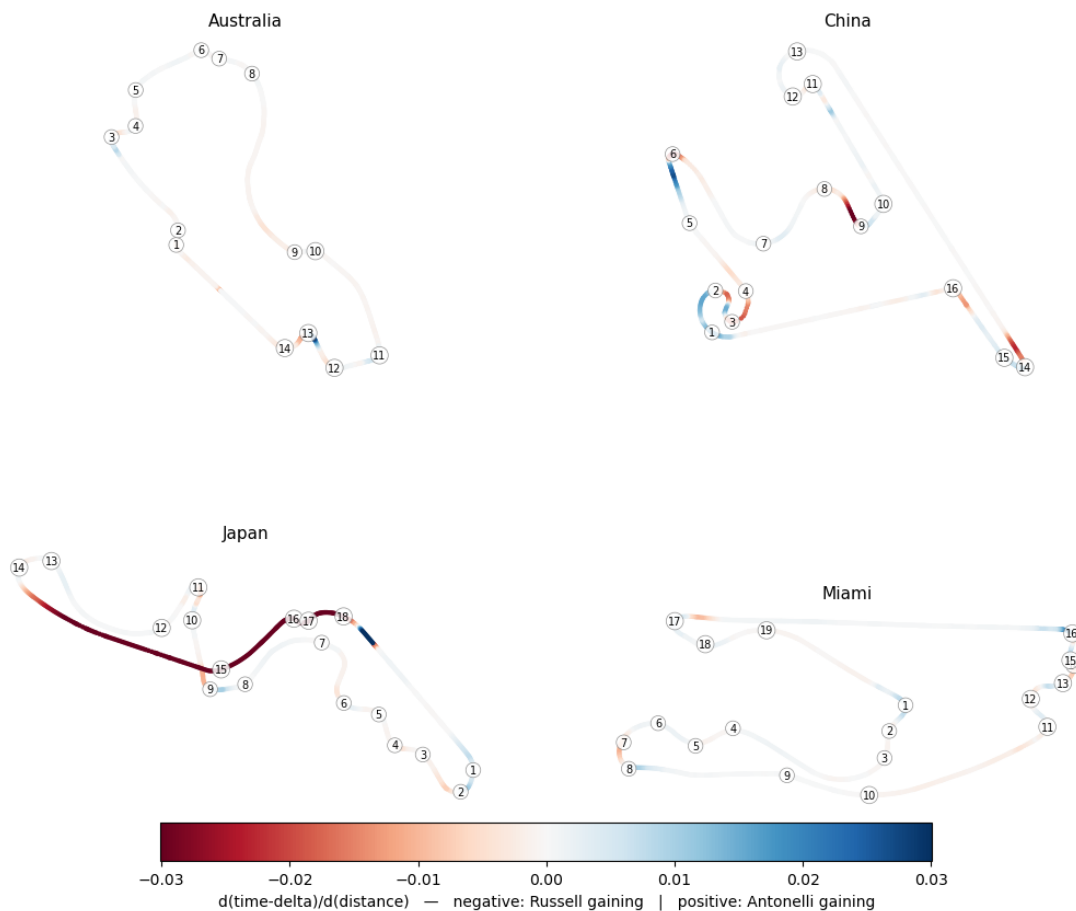
```

```

req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 22 drivers: ['12', '3', '16',
'1', '63', '44', '81', '43', '10', '27', '30', '87', '55', '31', '23', '41',
'14', '18', '77', '11', '5', '6']

```

Where Antonelli gains (blue) and where Russell gains (red)
Local rate of time gain/loss across the lap



1.7 How is he winning?

We know Antonelli is faster — at every track, every round. But how, specifically? The segment-level breakdown (above) turned out flat: straights, fast corners, slow corners, medium corners all essentially tied. So the advantage has to be hiding in something that segment-time averaging blurs over.

To find it, I looked at two specific signals at each corner on the lap. **Apex speed:** how much speed each driver carries through the deepest point of the corner. **Brake and throttle timing:** how late each driver brakes going in, and how soon they get back on throttle coming out.

1.7.1 Apex speed isn't where he's winning

```
[10]: corner_sigs = pd.concat([
    compute_corner_signatures(2026, race) for race in RACES
], ignore_index=True)
clean_corners = corner_sigs[corner_sigs['sensor_ok']].copy()

# Bucket by mean apex speed and report apex-speed delta
def speed_bucket(v):
    if v < 100: return 'very slow (<100)'
    if v < 150: return 'slow (100-150)'
    if v < 200: return 'medium (150-200)'
    return 'fast (>=200)'
clean_corners['speed_bucket'] = clean_corners['mean_apex_kph'].
    ↪apply(speed_bucket)
bucket_order = ['very slow (<100)', 'slow (100-150)', 'medium (150-200)', 'fast_
    ↪(>=200)']

apex_summary = (
    clean_corners.groupby('speed_bucket')['apex_delta_kph']
    .agg(['mean', 'count']).reindex(bucket_order)
    .rename(columns={'mean': 'mean ANT-RUS (kph)', 'count': 'n'})
)
print('Apex-speed delta by corner-speed bucket (positive = Antonelli faster at_
    ↪the apex):')
display(apex_summary.round(2))
```

```
core          INFO      Loading data for Australian Grand Prix - Qualifying
[v3.8.1]

req           INFO      Using cached data for session_info
req           INFO      Using cached data for driver_info
req           INFO      Using cached data for session_status_data
req           INFO      Using cached data for track_status_data
req           INFO      Using cached data for _extended_timing_data
req           INFO      Using cached data for timing_app_data
core          INFO      Processing timing data...
core          WARNING    No lap data for driver 18
core          WARNING    No lap data for driver 55
```

```

core          WARNING    Failed to perform lap accuracy check - all laps marked
as inaccurate (driver 18)

core          WARNING    Failed to perform lap accuracy check - all laps marked
as inaccurate (driver 55)

req           INFO       Using cached data for car_data

req           INFO       Using cached data for position_data

req           INFO       Using cached data for weather_data

req           INFO       Using cached data for race_control_messages

core          INFO       Finished loading data for 22 drivers: ['63', '12', '6',
'16', '81', '1', '44', '30', '41', '5', '27', '87', '31', '10', '23', '43',
'14', '11', '77', '18', '3', '55']

core          INFO       Loading data for Chinese Grand Prix - Qualifying
[v3.8.1]

req           INFO       Using cached data for session_info

req           INFO       Using cached data for driver_info

req           INFO       Using cached data for session_status_data

req           INFO       Using cached data for track_status_data

req           INFO       Using cached data for _extended_timing_data

req           INFO       Using cached data for timing_app_data

core          INFO       Processing timing data...

req           INFO       Using cached data for car_data

req           INFO       Using cached data for position_data

req           INFO       Using cached data for weather_data

req           INFO       Using cached data for race_control_messages

core          INFO       Finished loading data for 22 drivers: ['12', '63', '44',
'16', '81', '1', '10', '3', '6', '87', '27', '43', '31', '30', '41', '5', '55',
'23', '14', '77', '18', '11']

core          INFO       Loading data for Japanese Grand Prix - Qualifying
[v3.8.1]

req           INFO       Using cached data for session_info

req           INFO       Using cached data for driver_info

req           INFO       Using cached data for session_status_data

req           INFO       Using cached data for track_status_data

req           INFO       Using cached data for _extended_timing_data

req           INFO       Using cached data for timing_app_data

```

```

core          INFO      Processing timing data...
req           INFO      Using cached data for car_data
req           INFO      Using cached data for position_data
req           INFO      Using cached data for weather_data
req           INFO      Using cached data for race_control_messages
core          INFO      Finished loading data for 22 drivers: ['12', '63', '81',
'16', '1', '44', '10', '6', '5', '41', '3', '31', '27', '30', '43', '55', '23',
'87', '11', '77', '14', '18']
core          INFO      Loading data for Miami Grand Prix - Qualifying [v3.8.1]
req           INFO      Using cached data for session_info
req           INFO      Using cached data for driver_info
req           INFO      Using cached data for session_status_data
req           INFO      Using cached data for track_status_data
req           INFO      Using cached data for _extended_timing_data
req           INFO      Using cached data for timing_app_data
core          INFO      Processing timing data...
req           INFO      Using cached data for car_data
req           INFO      Using cached data for position_data
req           INFO      Using cached data for weather_data
req           INFO      Using cached data for race_control_messages
core          INFO      Finished loading data for 22 drivers: ['12', '3', '16',
'1', '63', '44', '81', '43', '10', '27', '30', '87', '55', '31', '23', '41',
'14', '18', '77', '11', '5', '6']

```

Apex-speed delta by corner-speed bucket (positive = Antonelli faster at the apex):

	mean ANT-RUS (kph)	n
speed_bucket		
very slow (<100)	-2.93	7
slow (100-150)	1.21	12
medium (150-200)	-0.58	14
fast (>=200)	2.76	30

At slow and medium corners — the ones with a real apex where commitment matters — Antonelli is tied or slightly behind Russell. He’s not carrying noticeably more speed through the deepest part of the corner. The narrow positive at fast corners is real but small, and many of those “corners” in FastF1’s circuit info are really high-speed kinks where the apex notion doesn’t apply cleanly.

So raw apex commitment isn’t the answer. The advantage has to come from somewhere else in the corner cycle.

1.7.2 He brakes later and gets back on throttle sooner — but only at fast corners

```
[11]: # Restrict to "real braking" corners: both drivers actually braked at least
# 20 m before the apex. Otherwise we're looking at flat-out kinks.
real = clean_corners[(clean_corners['brake_on_a'] > 20) &
    ↪(clean_corners['brake_on_b'] > 20)].copy()

bt_summary = (
    real.groupby('speed_bucket')[['brake_on_delta', 'throttle_full_delta']]
    .agg(['mean', 'count']).reindex(bucket_order)
    .round(1)
)
print('Brake-on Δ (negative = Antonelli brakes LATER)')
print('Throttle-full Δ (negative = Antonelli back on full throttle SOONER)')
display(bt_summary)

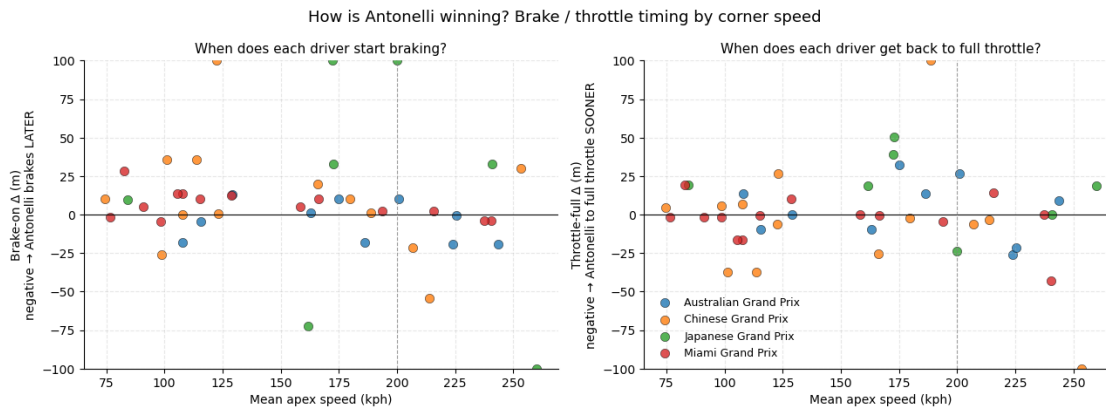
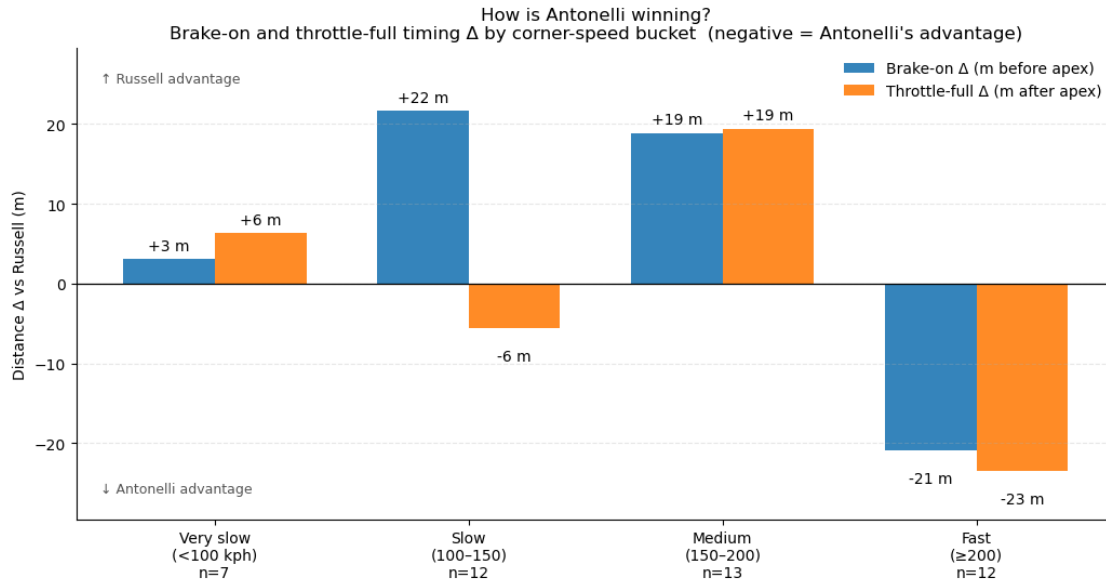
# Headline summary: per-bucket means as a grouped bar chart
fig = plot_corner_bucket_summary(corner_sigs, save_path='../figures/
    ↪corner_buckets.png')
plt.show()

# Per-corner detail: same data as a scatter so you can see the spread
fig = plot_corner_signatures(corner_sigs, save_path='../figures/
    ↪corner_signatures.png')
plt.show()
```

Brake-on Δ (negative = Antonelli brakes LATER)

Throttle-full Δ (negative = Antonelli back on full throttle SOONER)

	brake_on_delta		throttle_full_delta	
	mean	count	mean	count
speed_bucket				
very slow (<100)	3.1	7	6.3	7
slow (100-150)	21.6	12	-5.5	12
medium (150-200)	18.9	13	19.4	13
fast (>=200)	-20.9	12	-23.5	12



There it is. At fast corners — and only at fast corners — Antonelli brakes about 21 m later than Russell on average AND gets back to full throttle about 23 m sooner. That's the late-brake / early-throttle commitment signature you'd expect from a young driver willing to push the entry and exit of high-speed turns.

At slower corners his approach is the opposite: he brakes *earlier* and gets back on the gas *later* than Russell. It's almost like two different driving philosophies depending on speed range.

So the mechanism is: he wins at fast corners through commitment in the braking and traction zones, and gives a bit of time back at slow corners. The fast-corner gain wins out on net.

A real caveat: 44 corners across 4 races, only n=12 in the fast bucket, and a handful of dramatic outliers (Japan T8 alone has Russell braking 200 m earlier than Antonelli at a 260-kph kink). The fast-corner signature is consistent and large enough to feel real, but I'd want more clean races before calling it final.

1.8 Year-over-year: rookie vs sophomore

The within-2026 segment-level breakdown collapsed under the data-quality filter, but the lap-level comparison against 2025 — Antonelli’s actual rookie season at Mercedes — is far more striking. The cell below pulls the same 4 races from 2025 and plots both years on one chart.

At each of these tracks, Antonelli has gained between 0.3 and 0.7 s on Russell year-over-year. He’s not just continuing his rookie-year closing arc; he’s compressing it: he was almost a second behind Russell at his Australia debut and only just crossed ahead by Miami (R6) — in 2026 he started at -0.29 s and was ahead from R2 onwards.

```
[12]: # 2025: same 4 tracks (note: in the 2025 calendar Miami was R6, not R4)
results_2025 = {race: compare_teammates(2025, race) for race in RACES}

# Build the lap-delta arrays in calendar order
deltas_2026 = [results[r]['meta']['lap_delta_s'] for r in RACES]
deltas_2025 = [results_2025[r]['meta']['lap_delta_s'] for r in RACES]

# Flag known caveats per (year, track): 2025 Australia had a Q-session mismatch
# (ANT's fastest was in Q1, RUS's in Q3 - track-evolution likely inflates the
↳ gap).
flagged_2025 = [results_2025[r]['meta']['q_mismatch'] for r in RACES]
flagged_2026 = [results[r]['meta']['q_mismatch'] for r in RACES]

# Year-over-year deltas (positive = ANT gained vs Russell since rookie year)
yoy_gains = [d26 - d25 for d25, d26 in zip(deltas_2025, deltas_2026)]
summary = pd.DataFrame({
    'Track': RACES,
    '2025 Δ (s)': [round(d, 3) for d in deltas_2025],
    '2026 Δ (s)': [round(d, 3) for d in deltas_2026],
    'YoY gain (s)': [round(g, 3) for g in yoy_gains],
    '2025 Q-mismatch?': flagged_2025,
})
display(summary)
print(f'Mean YoY gain across the 4 tracks: {sum(yoy_gains)/len(yoy_gains):+.3f}↳
↳ s')

fig = plot_yoy_lap_deltas(
    RACES, deltas_2025, deltas_2026,
    flagged_2025=flagged_2025, flagged_2026=flagged_2026,
    save_path='../figures/year_over_year.png',
)
plt.show()
```

```
core          INFO      Loading data for Australian Grand Prix - Qualifying
[v3.8.1]
```

```
req           INFO      Using cached data for session_info
```

```
req           INFO      Using cached data for driver_info
```

```

req          INFO      Using cached data for session_status_data
req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
core         WARNING    Fixed incorrect tyre stint information for driver '4'
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 20 drivers: ['4', '81', '1',
'63', '22', '23', '16', '44', '10', '55', '6', '14', '18', '7', '5', '12', '27',
'30', '31', '87']
core         INFO      Loading data for Chinese Grand Prix - Qualifying
[v3.8.1]
req          INFO      Using cached data for session_info
req          INFO      Using cached data for driver_info
req          INFO      Using cached data for session_status_data
req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 20 drivers: ['81', '63', '4',
'1', '44', '16', '6', '12', '22', '23', '31', '27', '14', '18', '55', '10',
'87', '7', '5', '30']
core         INFO      Loading data for Japanese Grand Prix - Qualifying
[v3.8.1]
req          INFO      Using cached data for session_info
req          INFO      Using cached data for driver_info
req          INFO      Using cached data for session_status_data

```

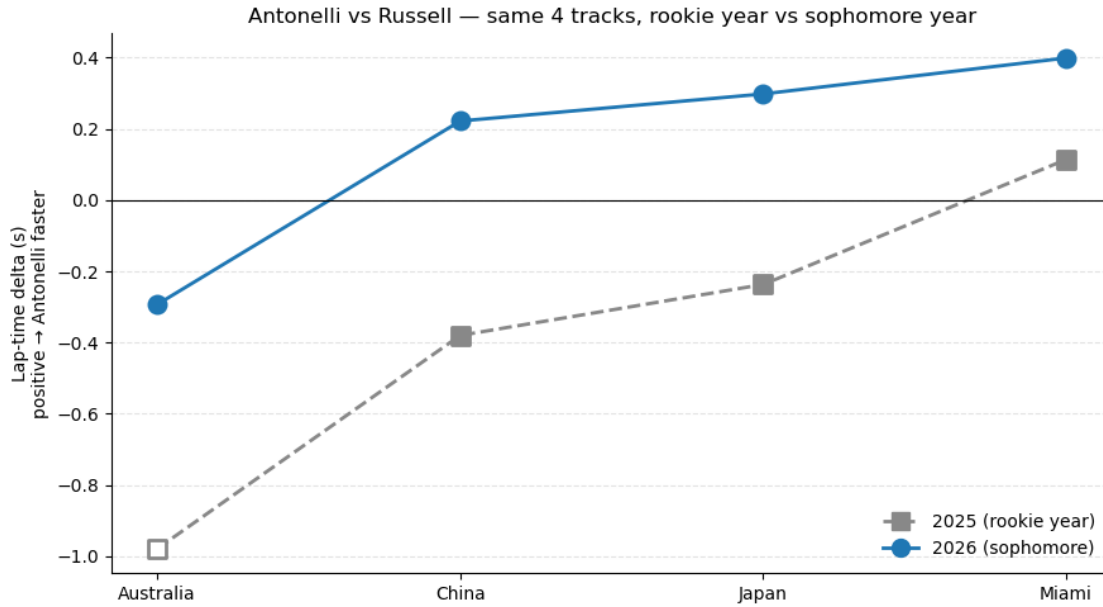
```

req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 20 drivers: ['1', '4', '81',
'16', '63', '12', '6', '44', '23', '87', '10', '55', '14', '30', '22', '27',
'5', '31', '7', '18']
core         INFO      Loading data for Miami Grand Prix - Qualifying [v3.8.1]
req          INFO      Using cached data for session_info
req          INFO      Using cached data for driver_info
req          INFO      Using cached data for session_status_data
req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 20 drivers: ['1', '4', '12',
'81', '63', '55', '23', '16', '31', '22', '6', '44', '5', '7', '30', '27', '14',
'10', '18', '87']

```

	Track	2025 Δ (s)	2026 Δ (s)	YoY gain (s)	2025 Q-mismatch?
0	Australia	-0.979	-0.293	0.686	True
1	China	-0.380	0.222	0.602	False
2	Japan	-0.237	0.298	0.535	False
3	Miami	0.114	0.399	0.285	False

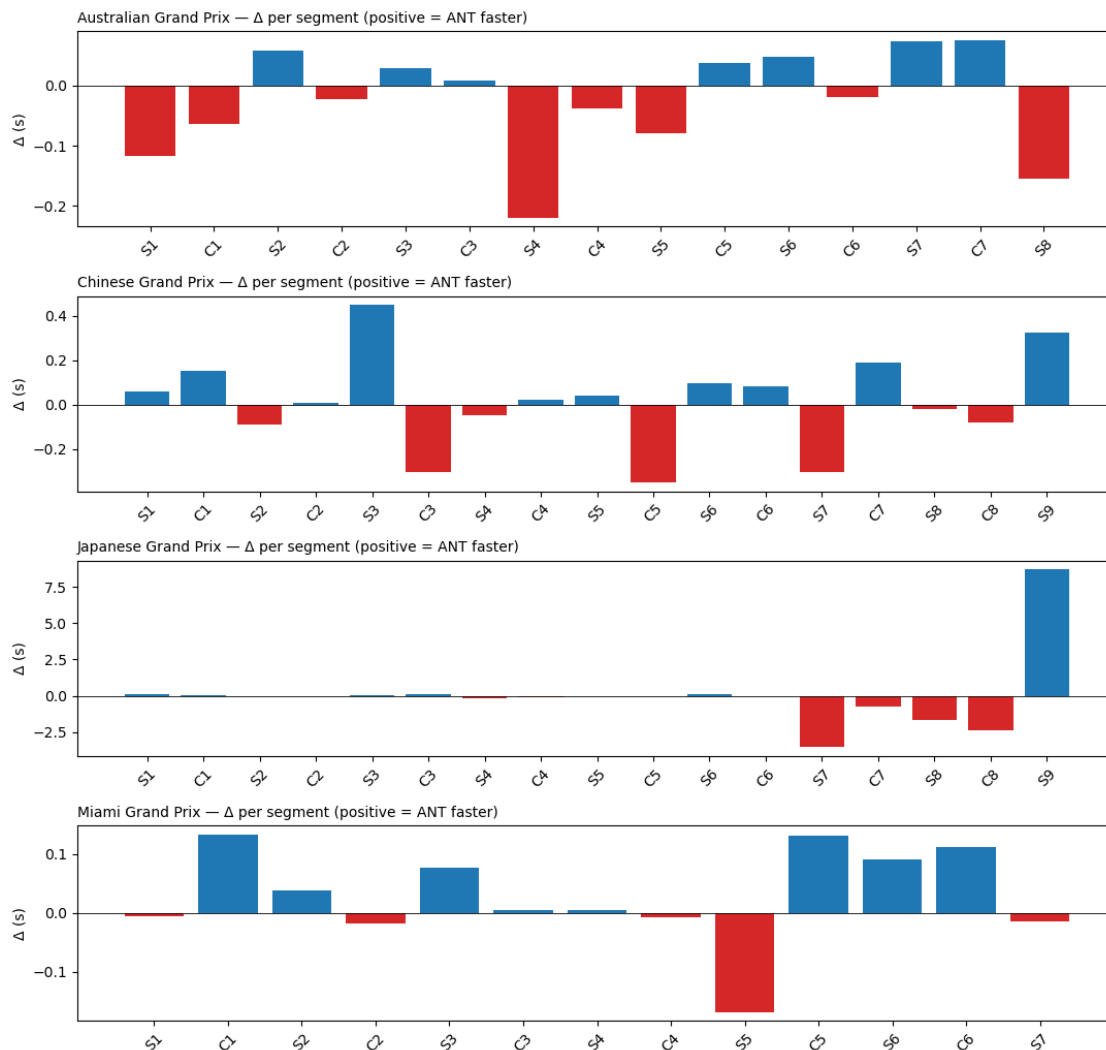
Mean YoY gain across the 4 tracks: +0.527 s



1.9 Per-race detail (appendix)

For each race, the raw per-segment delta. Useful for curious readers; not the main story.

```
[13]: fig, axes = plt.subplots(len(RACES), 1, figsize=(11, 2.6 * len(RACES)),
    ↪sharex=False)
    for ax, race in zip(axes, RACES):
        r = results[race]
        segs = r['segments'].sort_values('start_m')
        colors = ['#d62728' if d < 0 else '#1f77b4' for d in segs['delta_s']]
        ax.bar(segs['Segment'], segs['delta_s'], color=colors)
        ax.axhline(0, color='black', linewidth=0.6)
        ax.set_title(f"{r['meta']['event_name']} \u2014 \u2014 per segment
    ↪(positive = ANT faster)", loc='left', fontsize=10)
        ax.set_ylabel('\u0394 (s)')
        ax.tick_params(axis='x', rotation=45)
    fig.tight_layout()
    plt.show()
```



1.10 Caveats

- **Sample size: four races.** Findings are directional, not conclusive — and as the sensor investigation above shows, segment-level findings in particular are fragile to single-race data issues.
- **Q-session timing.** Q1 / Q2 / Q3 happen on an evolving track. The summary table flags any race where the two drivers' fastest valid laps came from different Q-segments.
- **Setup divergence.** Mercedes drivers do not always run identical setups. Public telemetry can't separate driver from setup.
- **Traffic and tire age within Q.** Out-laps, in-laps, and intra-session tire age all affect achievable lap time.
- **Telemetry sensor freezes.** FastF1's car telemetry occasionally stops reporting changes for long stretches. In this dataset, five Japan segments (S7, C7, S8, C8, S9 — covering D 4000 m to the end of the lap) are excluded by the sensor-quality filter above. Lap-level and sector-level deltas are unaffected — those come from timing-line beams, independent of the

car telemetry. Expect mild visual distortion at Japan’s T14–T18 in the track-delta map.

1.11 What I found

The strongest signal across four races is the **year-over-year comparison**. Antonelli has gained between 0.3 and 0.7 s on Russell at each of the same 4 tracks since his rookie season — mean **+0.53 s/track**. He started his rookie year nearly a full second behind Russell at Australia and only crossed ahead by Miami (R6); in 2026 he started at -0.29 s and was ahead from R2 onwards. The rookie-year closing arc is being compressed, not just sustained. Sector-time data, which comes from the timing-line beam independent of car telemetry, confirms each lap-level delta — including the Japan lap where the car’s speed sensor partially failed.

The simple segment-level breakdown turned out flat. My first pass reported a $+0.17$ s/lap straights advantage and a -0.46 s/lap medium-corner deficit for Antonelli, but both were almost entirely artifacts of five Japan segments where his speed sensor was frozen near 189 kph for over 1.3 km of the lap. Once a freeze-detection filter excludes them, all four categories collapse to within ± 0.01 s/lap of zero. That pushed me into the corner-cycle analysis above to find out where the lap time actually comes from.

The mechanism is specifically a fast-corner story. At corners with mean apex speeds ~ 200 kph, Antonelli brakes about 21 m later than Russell on average and gets back to full throttle about 23 m sooner — the late-brake / early-throttle commitment signature. At slower corners his approach is the opposite (earlier braking, later throttle), but the fast-corner gain wins out on net. The sample is small — 12 fast-corner data points across 4 races — but the pattern is consistent.

I’ll re-run the same framework after each 2026 race. See [What I’d build next](#) for the extensions.